Data Structures (CSN12101)
ASSIGNMENT-7

Name : Aayansh Niranjan Date :18/03/2026
Reg_No.: 20253004

Q1. The contents of a stack S are as follows:
Stack(S) 99 20 4 27
Index 0 1 2 3 4 5 6
The Stack can store a maximum of seven elements and the top pointer currently points at index 3.
Write a program to show the stack contents and indicate the position of the top pointer after each of the following stack operations:
o Push (S, 5) o Push (S, 7) o Pop (S) o Pop (S) o Pop (S)
Deduce the contents of an empty Queue after the execution of the following operations in sequence:
o Enqueue (6) o Enqueue (8) o Enqueue (–1) o Dequeue() o Enqueue (7) o Dequeue() o Dequeue()

A1. CODE:

```
#include <stdio.h>
#define MAX 7
void push(int S[], int *top, int x){ if(*top<MAX-1) S[++(*top)] = x; }
void pop(int *top){ if(*top>=0) (*top)--; }
void showS(int S[], int top){
    for(int i=0;i<=top;i++) printf("%d ",S[i]);
    printf("Top=%d\n",top);
}
void enqueue(int Q[], int *r, int x){
    if(*r==-1) Q[++(*r)] = x;
    else Q[++(*r)] = x;
}
void dequeue(int *f){ (*f)++; }
void showQ(int Q[], int f, int r){
    for(int i=f;i<=r;i++) printf("%d ",Q[i]);
    printf("\n");
}
int main(){
    int S[MAX]={99,20,4,27}, top=3;
    showS(S,top);
    push(S,&top,5); showS(S,top);
    push(S,&top,7); showS(S,top);
    pop(&top); showS(S,top);
    pop(&top); showS(S,top);
    pop(&top); showS(S,top);

    int Q[10], f=0, r=-1;
    enqueue(Q,&r,6);
    enqueue(Q,&r,8);
    enqueue(Q,&r,-1);
    dequeue(&f);
    enqueue(Q,&r,7);
    dequeue(&f);
    dequeue(&f);
    printf("Queue: ");
    showQ(Q,f,r);
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p1
99 20 4 27 Top=3
99 20 4 27 5 Top=4
99 20 4 27 5 7 Top=5
99 20 4 27 5 Top=4
99 20 4 27 Top=3
99 20 4 Top=2
Queue: 7
```

Q2. Write a C program to evaluate the prefix expression using Stack.
A2. CODE:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int s[20], top=-1;
void push(int x){ s[++top]=x; }
int pop(){ return s[top--]; }
int main(){
   char exp[]="+-7*45+20";
   for(int i=strlen(exp)-1;i>=0;i--){
      if(isdigit(exp[i])) push(exp[i]-'0');
      else{
         int a=pop(), b=pop();
         if(exp[i]=='+') push(a+b);
         else if(exp[i]=='-') push(a-b);
         else if(exp[i]=='*') push(a*b);
         else if(exp[i]=='/') push(a/b);
      }
   }
   printf("%d\n",pop());
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p2
-11
```

Q3. Write a C program to evaluate the postfix expression using Stack.
A3. CODE:

```
#include <stdio.h>
#include <ctype.h>

int s[20], top=-1;
void push(int x){ s[++top]=x; }
int pop(){ return s[top--]; }
int main(){
   char exp[]="56+7*8-";
   for(int i=0; exp[i]; i++){
      if(isdigit(exp[i])) push(exp[i]-'0');
      else{
         int b=pop(), a=pop();
         if(exp[i]=='+') push(a+b);
         else if(exp[i]=='-') push(a-b);
         else if(exp[i]=='*') push(a*b);
         else if(exp[i]=='/') push(a/b);
      }
   }
   printf("%d\n",pop());
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p3
69
```

Q4.Write a C program to evaluate the infix expression using Stack
A4. CODE:

```
#include <stdio.h>
#include <ctype.h>

int val[20], vtop=-1;
char op[20]; int otop=-1;
void pushVal(int x){ val[++vtop]=x; }
int popVal(){ return val[vtop--]; }
void pushOp(char x){ op[++otop]=x; }
char popOp(){ return op[otop--]; }
int apply(int a,int b,char op){
   if(op=='+') return a+b;
   if(op=='-') return a-b;
   if(op=='*') return a*b;
   if(op=='/') return a/b;
}
int prec(char c){
   if(c=='+'||c=='-') return 1;
   if(c=='*'||c=='/') return 2;
   return 0;
}
int main(){
   char exp[]="150*(20+10)/15";
   for(int i=0; exp[i]; i++){
      if(isdigit(exp[i])){
         int num=0;
         while(isdigit(exp[i])) num=num*10+(exp[i++]-'0');
         pushVal(num); i--;
      }
      else if(exp[i]=='(') pushOp(exp[i]);
      else if(exp[i]==')'){
         while(op[otop]!='('){
            int b=popVal(), a=popVal();
            pushVal(apply(a,b,popOp()));
         }
         popOp();
      }
      else{
         while(otop>=0 && prec(op[otop])>=prec(exp[i])){
            int b=popVal(), a=popVal();
            pushVal(apply(a,b,popOp()));
         }
         pushOp(exp[i]);
      }
   }
   while(otop>=0){
      int b=popVal(), a=popVal();
      pushVal(apply(a,b,popOp()));
   }
   printf("%d\n",popVal());
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p4
300
```

Q5. Write a C program to convert the infix expression to prefix expression.
A5. CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char c){ stack[++top] = c; }
char pop(){ return stack[top--]; }
char peek(){ return stack[top]; }
int precedence(char c){ return (c=='+'||c=='-')?1:(c=='*'||c=='/')?2:0; }
void infixToPrefix(char infix[], char prefix[]){
    int l = strlen(infix), k=0;
    char rev[MAX], temp;
    for(int i=0;i<l;i++) rev[i]=infix[l-i-1];
    rev[l]='\0';
    for(int i=0;i<l;i++){
        if(rev[i]=='(') rev[i]=')';
        else if(rev[i]==')') rev[i]='(';
    }
    for(int i=0;i<l;i++){
        if(isalnum(rev[i])) prefix[k++]=rev[i];
        else if(rev[i]=='(') push('(');
        else if(rev[i]==')'){
            while(top!=-1 && peek()!='(') prefix[k++]=pop();
            pop();
        }else{
            while(top!=-1 && precedence(peek())>=precedence(rev[i])) prefix[k++]=pop();
            push(rev[i]);
        }
    }
    while(top!=-1) prefix[k++]=pop();
    prefix[k]='\0';
    for(int i=0;i<k/2;i++){ temp=prefix[i]; prefix[i]=prefix[k-i-1]; prefix[k-i-1]=temp; }
}
int main(){
    char infix[]="(A+B)*(C+D)", prefix[MAX];
    infixToPrefix(infix, prefix);
    printf("Prefix: %s\n", prefix);
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p5
Prefix: *+AB+CD
```

Q6. Write a C program to convert the infix expression to postfix expression.

A6. CODE:

```
#include <stdio.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
char peek() { return stack[top]; }
int precedence(char c) {
    if(c=='+'||c=='-') return 1;
    if(c=='*'||c=='/') return 2;
    return 0;
}
void infixToPrefix(char *infix, char *prefix) {
    int i, k=0, len = strlen(infix);
    char rev[MAX], temp;
    for(i=0;i<len;i++){
        temp = infix[len-1-i];
        rev[i] = (temp=='(')?')':(temp==')'?'(':temp);
    }
    rev[len] = '\0';
    for(i=0;i<len;i++){
        if(isalnum(rev[i]))
            prefix[k++] = rev[i];
        else if(rev[i]=='(')
            push(rev[i]);
        else if(rev[i]==')'){
            while(top!=-1 && peek()!='(')
                prefix[k++] = pop();
            pop();
        } else {
            while(top!=-1 && precedence(peek())>=precedence(rev[i]))
                prefix[k++] = pop();
            push(rev[i]);
        }
    }
    while(top!=-1) prefix[k++] = pop();
    prefix[k]='\0';
    for(i=0;i<k/2;i++){
        temp = prefix[i];
        prefix[i] = prefix[k-1-i];
        prefix[k-1-i] = temp;
    }
}
int main() {
    char infix[MAX], prefix[MAX];
    printf("Enter infix expression: ");
    scanf("%s", infix);
    infixToPrefix(infix, prefix);
    printf("Prefix expression: %s\n", prefix);
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p6
AB+CD+*
```

Q7. Write a C program that accepts a string and reverse it using a Stack.

A7. CODE:

```
#include <stdio.h>
#include <string.h>

char st[50]; int top=-1;
void push(char c){ st[++top]=c; }
char pop(){ return st[top--]; }
int main(){
    char str[50];
    scanf("%s", str);
    for(int i=0; str[i]; i++) push(str[i]);
    for(int i=0; i<strlen(str); i++) str[i]=pop();
    printf("%s\n", str);
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p7
Data_Structures
serutcurtS_ataD
```

Q8. Write a program to implement two stacks in a single array.

A8. CODE:

```
#include <stdio.h>
#define MAX 10
int arr[MAX], top1=-1, top2=MAX;
void push1(int x){ if(top1<top2-1) arr[++top1]=x; }
void push2(int x){ if(top1<top2-1) arr[--top2]=x; }
int pop1(){ return arr[top1--]; }
int pop2(){ return arr[top2++]; }
int main(){
    push1(10); push1(20);
    push2(100); push2(200);
    printf("%d %d\n", pop1(), pop2());
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p8
20 200
```

Q9. Write a program to check the valid parenthesis in a given expression.
A9. CODE:

```
#include <stdio.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
char peek() { return stack[top]; }
int isMatching(char open, char close) {
    return (open=='(' && close==')') ||
           (open=='{' && close=='}') ||
           (open=='[' && close==']');
}
int isBalanced(char *expr) {
    int i;
    for(i=0; expr[i]; i++) {
        if(expr[i]=='(' || expr[i]=='{' || expr[i]=='[')
            push(expr[i]);
        else if(expr[i]==')' || expr[i]=='}' || expr[i]==']') {
            if(top==-1 || !isMatching(pop(), expr[i]))
                return 0; // unbalanced
        }
    }
    return top==-1; // true if stack is empty
}
int main() {
    char expr[256];
    printf("Enter expression: ");
    fgets(expr, sizeof(expr), stdin);
    expr[strcspn(expr, "\n")] = 0; // remove newline
    if(isBalanced(expr))
        printf("Parentheses are balanced.\n");
    else
        printf("Parentheses are NOT balanced.\n");
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p9
Enter expression: ((a-b/c)*(a/k-l))
Parentheses are balanced.
```

Q10.Write a program for Tower of Hanoi problem using stack.
A10. CODE:

```
#include <stdio.h>
void hanoi(int n, char A, char B, char C){
    if(n==1){
        printf("Move %c -> %c\n", A, C);
        return;
    }
    hanoi(n-1, A, C, B);
    printf("Move %c -> %c\n", A, C);
    hanoi(n-1, B, A, C);
}
int main(){
    int n=3;
    hanoi(n,'A','B','C');
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p10
Move A -> C
Move A -> B
Move C -> B
Move A -> C
Move B -> A
Move B -> C
Move A -> C
```

Q11. Write a program to implement Circular Queue.
a. Insert an element into the circular queue.
b. Delete an element from the circular queue.
c. Display the contents of the circular queue.
A11. CODE:

```
#include <stdio.h>
#define N 5

int cq[N], front = -1, rear = -1;
void insert(int x) {
    if ((rear + 1) % N == front)
        printf("Overflow\n");
    else {
        if (front == -1) front = 0;
        rear = (rear + 1) % N;
        cq[rear] = x;
    }
}
void delete() {
    if (front == -1)
        printf("Underflow\n");
    else {
        printf("Deleted: %d\n", cq[front]);
        if (front == rear)
            front = rear = -1;
        else
            front = (front + 1) % N;
    }
}
void display() {
    if (front == -1) {
        printf("Empty\n");
        return;
    }
    int i = front;
    while (1) {
        printf("%d ", cq[i]);
        if (i == rear) break;
        i = (i + 1) % N;
    }
    printf("\n");
}
int main() {
    insert(10); insert(20); insert(30);
    display();
    delete();
    display();
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p11
10 20 30
Deleted: 10
20 30
```

Q12. Write a program to implement Priority Queue.
a. Insert an element into the priority queue.
b. Delete an element from the priority queue.
c. Display the contents of the priority queue.
A12. CODE:

```
#include <stdio.h>
#define N 5

int pq[N], size = 0;
void insert(int x) {
    if (size == N)
        printf("Overflow\n");
    else {
        int i = size - 1;
        while (i >= 0 && pq[i] < x) {
            pq[i + 1] = pq[i];
            i--;
        }
        pq[i + 1] = x;
        size++;
    }
}
void delete() {
    if (size == 0)
        printf("Underflow\n");
    else
        printf("Deleted: %d\n", pq[--size]);
}
void display() {
    if (size == 0) {
        printf("Empty\n");
        return;
    }
    for (int i = 0; i < size; i++)
        printf("%d ", pq[i]);
    printf("\n");
}
int main() {
    insert(30); insert(10); insert(50);
    display();
    delete();
    display();
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p12
50 30 10
Deleted: 10
50 30
```

Q13. Write a C program to delete the nth element of a queue. Firstly, this code checks if the queue is empty or if the position to delete is invalid. If the position is valid, it deletes the first n-1 elements, then deletes the nth element by calling the dequeue() function.

A13. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int queue[MAX], front = -1, rear = -1;
void enqueue(int x) {
    if (rear == MAX - 1){printf("Queue Overflow\n");return;}
    if (front == -1) front = 0;
    queue[++rear] = x;
}
int dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return -1;
    }
    return queue[front++];
}
void display() {
    if (front == -1 || front > rear){printf("Queue is empty\n");return;}
    for (int i = front; i <= rear; i++) printf("%d ", queue[i]);
    printf("\n");
}
void deleteNth(int n) {
    int size = rear - front + 1;
    if (front == -1 || size == 0){printf("Error: Queue is empty\n"); return;}
    if (n <= 0 || n > size) {printf("Error: Invalid position\n"); return;}
    int temp[MAX], k = 0;
    for (int i = 1; i < n; i++) temp[k++] = dequeue();
    int deleted = dequeue();
    for (int i = 0; i < k; i++) enqueue(temp[i]);
    printf("Deleted element: %d\n", deleted);
}
int main() {
    printf("Insert some elements into the queue:\n");
    enqueue(1);
    enqueue(2);
    enqueue(3);
    enqueue(4);
    enqueue(5);
    printf("Queue elements are: ");
    display();
    printf("Delete the 7th element of the said queue:\n");
    deleteNth(7);
    printf("Delete the 3rd element of the said queue:\n");
    deleteNth(3);
    printf("Queue elements are: ");
    display();
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_7$ ./p13
Insert some elements into the queue:
Queue elements are: 1 2 3 4 5
Delete the 7th element of the said queue:
Error: Invalid position
Delete the 3rd element of the said queue:
Deleted element: 3
Queue elements are: 4 5 1 2
```